

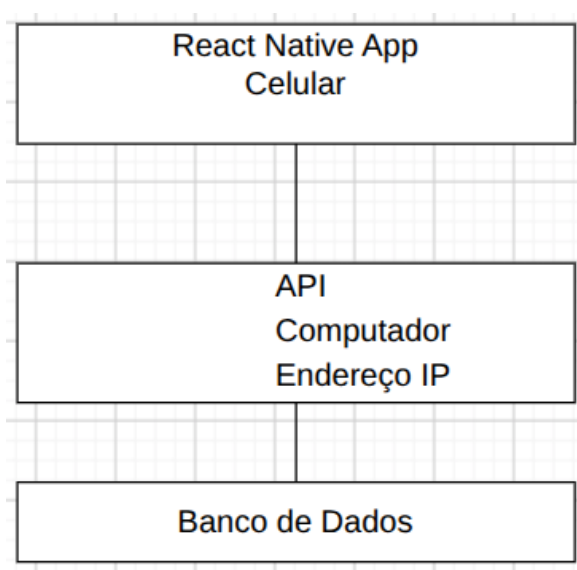
Guia de desenvolvimento com o React Native

Prof: Vivianne Melo

O React Native (RN) é um framework baseado em JavaScript que permite desenvolver aplicativos para Android e iOS utilizando uma única base de código. Criado pelo Facebook em 2015, ele facilita o desenvolvimento multiplataforma, reduzindo tempo e custos de manutenção.

Ele é baseado no React, aproveitando conceitos como componentes, estados e propriedades. Por permitir o compartilhamento de grande parte do código entre diferentes plataformas, tornou-se uma opção bastante utilizada no desenvolvimento de aplicativos mobile.

Arquitetura da aplicação que faremos



Após a finalização das **APIs**, cada grupo iniciará a preparação do ambiente para a construção do aplicativo

Preparação do ambiente

Vamos utilizar no celular o aplicativo **Expo Go**, então, já instale ele no seu celular android.

Verifique se o seu computador possui o Node.js instalado e o

NPM-Node Package Manager (Gerenciador de Pacotes do Node) instalados utilizando os seguintes comandos:

node --version

npm --version

Criando projeto em branco no visual studio code

Primeiro acesse o seu projeto no visual studio code e abra o terminal e depois digite o seguinte comando:

npx create-expo-app@latest meu-app --template blank

No lugar do meu-app insira o nome do seu aplicativo, o template blank realiza a criação de um template totalmente limpo, apenas com o JavaScript mínimo.

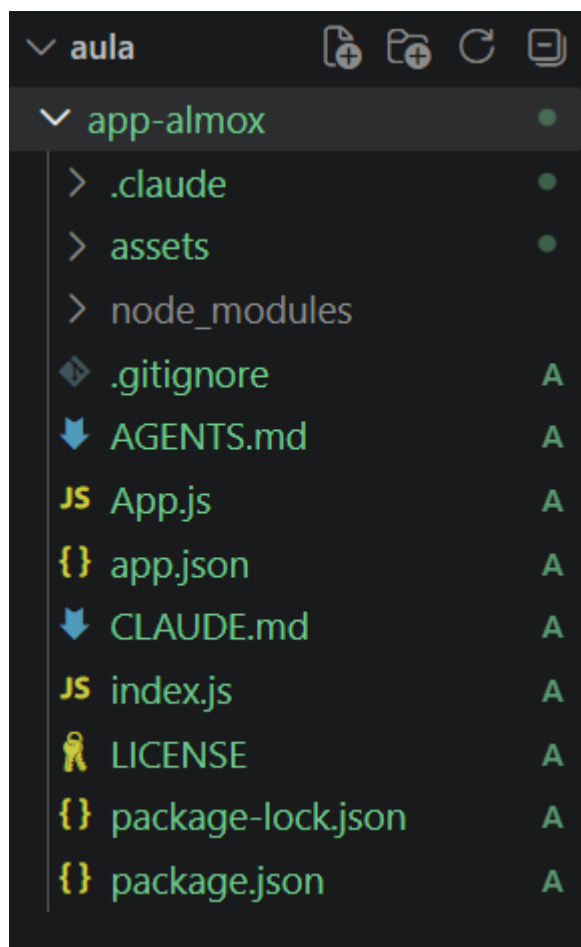
Depois selecione o **SDK**(Software Development Kit), ele contém Ferramentas de Build, Debuggers, Bibliotecas, Documentação, um conjunto de ferramentas necessárias para o desenvolvimento do aplicativo

```
PS C:\Users\vivia\Desktop\aula> npx create-expo-app@latest app-almox --template blank
Need to install the following packages:
create-expo-app@4.0.0
Ok to proceed? (y)

? Select an Expo SDK version: » - Use arrow-keys. Return to submit.
> Latest (SDK 57) - Recommended for most projects
Other SDK version...
```

Após toda a instalação do SDK você verá que no seu IDE aparecerá a pasta contendo arquivos necessários para o desenvolvimento do seu aplicativo

Exemplo



Acessando a pasta do seu aplicativo

Lembre-se no seu projeto existem diversos arquivos, para acessar a pasta do seu aplicativo utilize: **CD nome-do-aplicativo**

Exemplo

```
PS C:\Users\vivia\Desktop\aula> npx create-expo-app@latest app-a
- npm run web
npm notice
npm notice New minor version of npm available! 11.6.1 -> 11.19.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.19.1
npm notice To update run: npm install -g npm@11.19.1
npm notice
PS C:\Users\vivia\Desktop\aula> cd app-almox
PS C:\Users\vivia\Desktop\aula\app-almox>
```

Iniciando o aplicativo

Utilize o comando para iniciar o serviço do aplicativo:

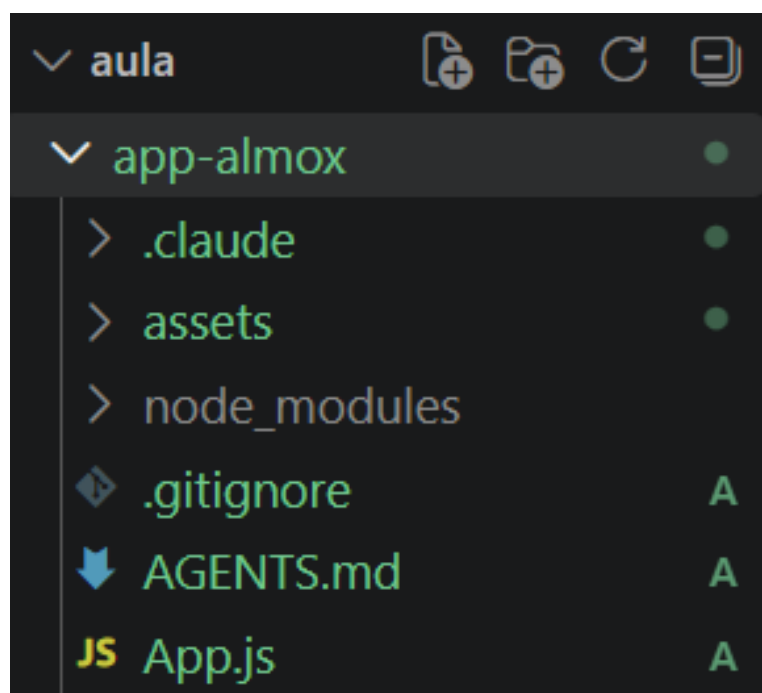
npx expo start

No final dele aparecerá um **QR Code** que você poderá realizar a leitura no expo go do seu celular



Criando primeira Olá Turma

O arquivo **App.js** será o nosso ponto de partida, acesse primeiro ele para começarmos a montar a interface.



Exemplo

```

app-almox > js App.js > App
1  import { StatusBar } from 'expo-status-bar';
2  import { StyleSheet, Text, View } from 'react-native';
3
4  export default function App() {
5    return (
6      <View style={styles.container}>
7        <Text>Oi Turma!</Text>
8        <StatusBar style="auto" />
9      </View>
10    );
11  }
12
13  const styles = StyleSheet.create({
14    container: {
15      flex: 1,
16      backgroundColor: '■ #fff',
17      alignItems: 'center',
18      justifyContent: 'center',
19    },
20  });

```

Antes de trabalharmos com o endpoint da API, precisamos aprender a guardar os dados que o usuário digita no aplicativo.

No React Native, utilizamos o **useState** para armazenar e atualizar essas informações.

Guardando o produto

Primeiro, vamos criar um estado para guardar o nome do produto.

Exemplo: `const [produto, setProduto] = useState('');`

Nesse exemplo estamos criando um estado **produto**, onde guardamos o valor atual.

- **produto** guarda o valor atual.
- O **setProduto** realiza a alteração do valor

- E as aspas simples são o valor inicial, começa vazio (' ')

Por exemplo, para alterar o produto para Martelo, podemos fazer:

```
setProduto('Martelo');
```

Dessa forma o valor do produto será Martelo.

Criando a tela de cadastro

Pense em uma tela que tenha produto, quantidade e um botão de cadastro.

- Primeiro criamos o estado produto:

```
const [produto, setProduto] = useState("");
```

- Depois criamos o estado quantidade:

```
const [quantidade, setQuantidade] = useState("");
```

Agora podemos adicionar os campos na tela.

Para o produto:

```
<TextInput  
  value={produto}  
  onChangeText={setProduto}  
  placeholder="Digite o produto"  
/>
```

Para a quantidade:

```
<TextInput  
  value={quantidade}  
  onChangeText={setQuantidade}  
  placeholder="Digite a quantidade"  
/>
```

Dessa forma, o que o usuário digitar será armazenado nos estados produto e quantidade.

Enviando os dados para a API

Depois que o usuário preencher os campos, teremos, por exemplo:

Produto: Lápis

Quantidade: 20

O aplicativo está mantendo essas informações nos estados `produto` e `quantidade`. Para enviar os dados para a API, primeiro precisamos juntar as informações em um único objeto.

Podemos fazer isso da seguinte forma:

```
const dados = {  
  produto: produto,  
  quantidade: quantidade  
};
```

Dessa forma teremos um **objeto** com os dados preenchidos:

```
{  
  produto: "Lápis",  
  quantidade: "20"  
}
```

Transformando os dados em JSON

Antes de enviar os dados para a API, podemos transformar o objeto em JSON utilizando `JSON.stringify()`:

Exemplo

```
const json = JSON.stringify(dados);
```

Observação: O **useState** é responsável por guardar os dados que o usuário está preenchendo no aplicativo.

O objeto **dados** junta essas informações.

O **JSON.stringify()** transforma o objeto em JSON, deixando os dados prontos para serem enviados.

Exemplo completo Tela de Cadastro

```
import { useState } from 'react';
import { StyleSheet, Text, TextInput, View, Button } from
'react-native';

export default function App() {
  const [produto, setProduto] = useState('');
  const [quantidade, setQuantidade] = useState('');

  function cadastrar() {
    console.log({ produto, quantidade });
  }

  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Cadastro de Produto</Text>

      <Text>Produto</Text>
      <TextInput
        style={styles.input}
        value={produto}
        onChangeText={setProduto}
        placeholder="Nome do produto"
      />

      <Text>Quantidade</Text>
      <TextInput
        style={styles.input}
        value={quantidade}
        onChangeText={setQuantidade}
        placeholder="Quantidade"
        keyboardType="numeric"
      />
    </View>
  );
}
```

```

        <Button title="Cadastrar" onPress={cadastrar} />
      </View>
    );
  }

  const styles = StyleSheet.create({
    container: {
      flex: 1,
      padding: 20,
      justifyContent: 'center',
    },
    titulo: {
      fontSize: 24,
      fontWeight: 'bold',
      textAlign: 'center',
      marginBottom: 30,
    },
    input: {
      borderWidth: 1,
      padding: 10,
      marginBottom: 20,
    },
  });

```

Para integrar com a API vamos utilizar uma função **fetch**.

Enviando os dados com fetch

O `fetch()` é uma função do JavaScript utilizada para realizar requisições para um endereço. No nosso caso, vamos utilizá-lo para enviar os dados para a API.

Para isso, precisamos transformar a função `cadastrar()` em uma função assíncrona:

```
async function cadastrar() {
```

Depois, utilizamos o `fetch()`:

```
const dados = { produto, quantidade };
console.log(dados);
const resposta = await fetch(
  'http://IP_DO_COMPUTADOR:5001/api/cadastro',
  {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(dados),
  }
);
```

Detalhes

- **fetch()**: Utilizamos para realizar a requisição para a API.
- **method**: 'POST'. utilizamos para indicar o método que estamos enviando dados.
- **headers**: Utilizados para informar que os dados estão no formato JSON.
- **body**: Contém os dados que serão enviados.
- **JSON.stringify(dados)**: Utilizamos para transformar o objeto em JSON.
- **await**: Aguarda a API responder.

Configurando o endereço da API

No código temos:

```
'http://IP_DO_COMPUTADOR:5001/api/cadastro',
```

Substitua `IP_DO_COMPUTADOR` pelo endereço IP do computador onde a API está sendo executada.

Por exemplo:

'http://192.168.0.11:5001/api/cadastro'

OBSERVAÇÃO: O celular e o computador devem estar conectados à mesma rede, quando essa for a configuração utilizada no projeto.

App.js integrado com a API

```
import { useState } from 'react';
import { StyleSheet, Text, TextInput, View, Button } from
'react-native';

export default function App() {
  const [produto, setProduto] = useState('');
  const [quantidade, setQuantidade] = useState('');

  async function cadastrar() {
    const dados = { produto, quantidade };

    console.log(dados);

    const resposta = await fetch(
      'http://IP_DO_COMPUTADOR:5001/api/cadastro',
      {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify(dados),
      }
    );

    console.log(await resposta.json());
  }

  return (
```

```

    <View style={styles.container}>
      <Text style={styles.titulo}>Cadastro de
Produto</Text>

      <Text>Produto</Text>
      <TextInput
        style={styles.input}
        value={produto}
        onChangeText={setProduto}
        placeholder="Nome do produto"
      />

      <Text>Quantidade</Text>
      <TextInput
        style={styles.input}
        value={quantidade}
        onChangeText={setQuantidade}
        placeholder="Quantidade"
        keyboardType="numeric"
      />

      <Button title="Cadastrar" onPress={cadastrar} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    justifyContent: 'center',
  },
  titulo: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 30,
  },
});

```

```
input: {  
  borderWidth: 1,  
  padding: 10,  
  marginBottom: 20,  
},  
));
```

Dica: Para estilizar vocês podem utilizar online no site
<https://snack.expo.dev/>